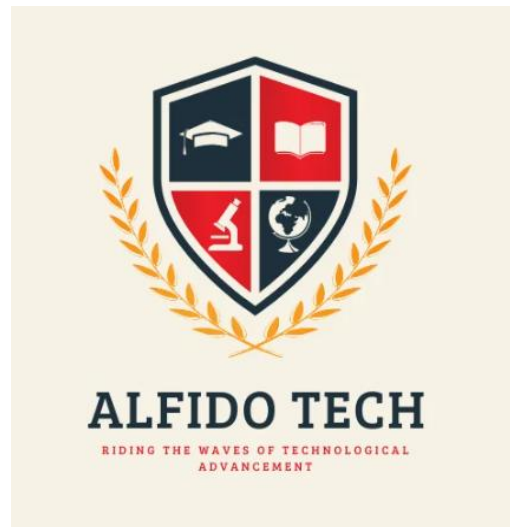


ALFIDO TECH

Java Developer Internship



TASK 1 REPORT

Java Basic & OOP Fundamentals

Submitted By: Alquama Arsalan Khan

Submission Date: 23 June 2026

Objective

The objective of this task is to understand the fundamentals of Java programming and Object-Oriented Programming (OOP) concepts. Through this task, various Java programs were developed to demonstrate the use of conditions, loops, arrays, classes, objects, inheritance, polymorphism, and encapsulation.

This task aims to build a strong foundation in Java by applying theoretical concepts through practical implementation. It also helps in improving problem-solving skills, logical thinking, and understanding of how real-world applications are designed using object-oriented principles.

By completing this task, I gained hands-on experience in writing Java programs and learned how OOP concepts help in creating efficient, reusable, and maintainable code.

This task also enhanced my understanding of software development practices and strengthened my programming skills for future Java-based projects.

Program 1:

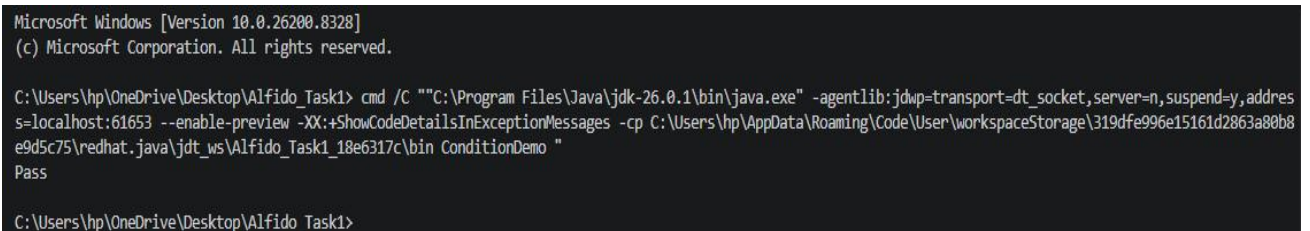
ConditionDemo.java

Code:



```
ConditionDemo.java X LoopDemo.java ArrayDemo.java
ConditionDemo.java > ConditionDemo
1 public class ConditionDemo {
    Run | Debug
2     public static void main(String[] args){
3         int marks=80;
4         if(marks>=50){
5             System.out.println(x: "Pass");
6         } else{
7             System.out.println(x: "Fail");
8         }
9     }
10 }
11 }
```

Output:



```
Microsoft Windows [Version 10.0.26200.8328]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp\OneDrive\Desktop\Alfido_Task1> cmd /C ""C:\Program Files\Java\jdk-26.0.1\bin\java.exe" -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:61653 --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\hp\AppData\Roaming\Code\User\workspaceStorage\319dfe996e15161d2863a80b8e9d5c75\redhat.java\jdt_ws\Alfido_Task1_18e6317c\bin ConditionDemo "
Pass

C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
```

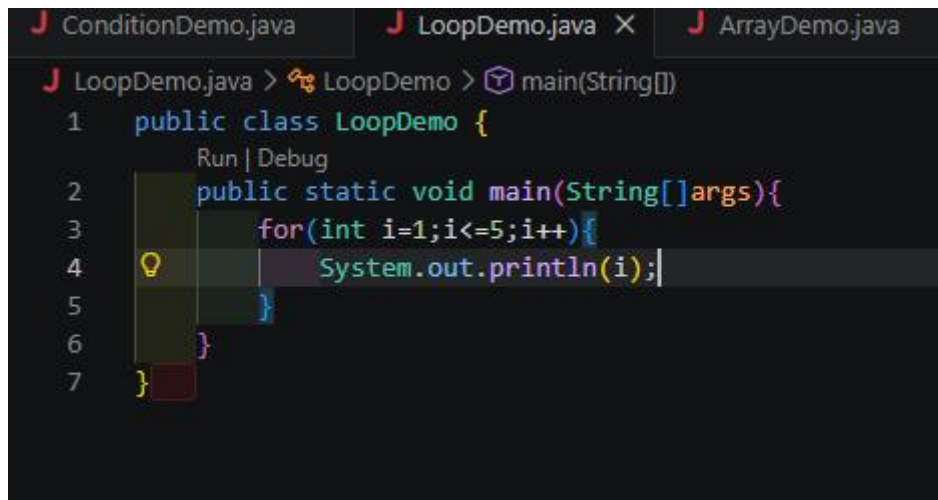
Explanation:

This program demonstrates the use of conditional statements (if-else). It checks whether the marks are greater than or equal to 50. If the condition is true, it displays "Pass"; otherwise, it displays "Fail".

Program 2:

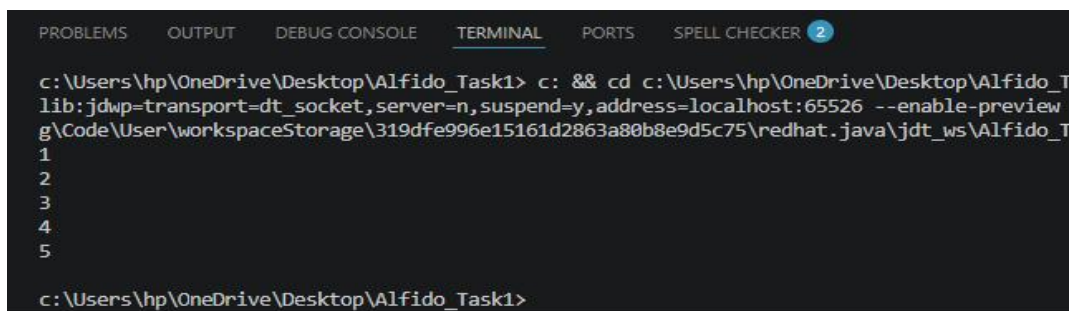
LoopDemo.java

Code:



```
ConditionDemo.java  LoopDemo.java X  ArrayDemo.java
J LoopDemo.java > LoopDemo > main(String[])
1 public class LoopDemo {
    Run | Debug
2     public static void main(String[] args){
3         for(int i=1;i<=5;i++){
4             System.out.println(i);
5         }
6     }
7 }
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER 2
c:\Users\hp\OneDrive\Desktop\Alfido_Task1> c: && cd c:\Users\hp\OneDrive\Desktop\Alfido_Task1
lib:jdw=transport=dt_socket,server=n,suspend=y,address=localhost:65526 --enable-preview
g\Code\User\workspaceStorage\319dfe996e15161d2863a80b8e9d5c75\redhat.java\jdt_ws\Alfido_Task1
1
2
3
4
5
c:\Users\hp\OneDrive\Desktop\Alfido_Task1>
```

Explanation:

This program demonstrates the use of a for loop. The loop starts from 1 and continues until 5, printing each number one by one.

Program 3:

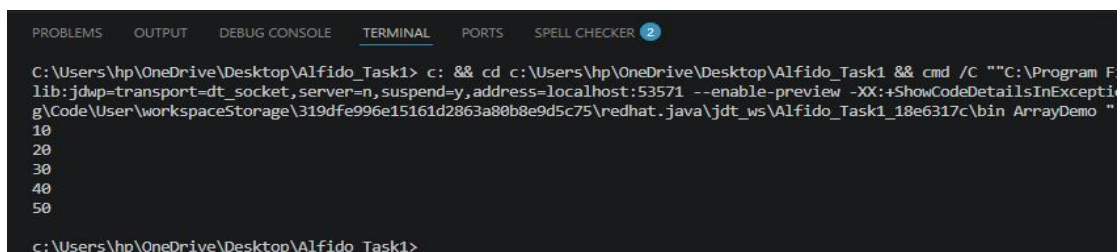
ArrayDemo.java

Code:



```
ConditionDemo.java  LoopDemo.java  ArrayDemo.java
ArrayDemo.java
1  public class ArrayDemo {
    Run | Debug
2      public static void main(String[] args) {
3          int[] numbers = {10, 20, 30, 40, 50};
4          for(int num : numbers){
5              System.out.println(num);
6          }
7      }
8  }
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER 2
C:\Users\hp\OneDrive\Desktop\Alfido_Task1> c: && cd c:\Users\hp\OneDrive\Desktop\Alfido_Task1 && cmd /C ""C:\Program F
lib:jdpw=transport=dt_socket,server=n,suspend=y,address=localhost:53571 --enable-preview -XX:+ShowCodeDetailsInExcepti
g\Code\User\workspaceStorage\319dfe996e15161d2863a80b8e9d5c75\redhat.java\jdt_ws\Alfido_Task1_18e6317c\bin ArrayDemo "
10
20
30
40
50
c:\Users\hp\OneDrive\Desktop\Alfido_Task1>
```

Explanation:

This program demonstrates the use of arrays in Java. An array is used to store multiple values of the same data type, and each value is displayed using a loop.

Program 4:

Classes and Objects

Code:

```
J Main.java > Main
1  class Student {
2      String name;
3      void display() {
4          System.out.println(name);
5      }
6  }
7  public class Main {
8      public static void main(String[] args) {
9          Student s1= new Student();
10         s1.name = "Ramsha";
11         s1.display();
12     }
13 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER 2
c:\Users\hp\OneDrive\Desktop\Alfido_Task1>
c:\Users\hp\OneDrive\Desktop\Alfido_Task1>
c:\Users\hp\OneDrive\Desktop\Alfido_Task1> c: && cd c:\Users\hp\OneDrive\Desktop\Alfido_Task1 && cmd /C ""C:\Program Files\Java\jdk-9.0.4\bin\java.exe -Xmx1024m -Xms128m -Djdt_ws=c:\Users\hp\OneDrive\Desktop\Alfido_Task1\jdt_ws\Alfido_Task1_18e6317c\bin Main "
Ramsha
c:\Users\hp\OneDrive\Desktop\Alfido_Task1>
```

Explanation:

This program demonstrates the concept of classes and objects. A class named Student is created, and an object of that class is used to access its properties and methods.

Program 5:

InheritanceDemo.java

Code:

```
InheritanceDemo.java
1  class Animal {
2
3      void sound() {
4          System.out.println(x: "Animal makes sound");
5      }
6
7  }
8
9  class Dog extends Animal {
10
11      void bark() {
12          System.out.println(x: "Dog barks");
13      }
14
15  }
16
17  public class InheritanceDemo {
18
19      Run | Debug
20      public static void main(String[] args) {
21
22          Dog d = new Dog();
23
24          d.sound();
25          d.bark();
26      }
27  }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER 2
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
C:\Users\hp\OneDrive\Desktop\Alfido_Task1> c: && cd c:\Users\hp\OneDrive\Desktop\Alfido_Task1 && cmd /C ""C:\Program Files\Java\lib\jdpw=transport=dt_socket,server=n,suspend=y,address=localhost:51233 --enable-preview -XX:+ShowCodeDetailsInExceptionMessage\Code\User\workspaceStorage\319dfe996e15161d2863a80b8e9d5c75\redhat.java\jdt_ws\Alfido_Task1_18e6317c\bin InheritanceDemo ""
Animal makes sound
Dog barks
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>]
```

Explanation:

This program demonstrates inheritance. The Dog class inherits properties and methods from the Animal class, allowing code reuse and establishing a parent-child relationship between classes.

Program 6:

PolymorphismDemo.java

Code:

```
PolymorphismDemo.java > PolymorphismDemo
1  class Animal {
2
3      void sound() {
4          System.out.println(x: "Animal sound");
5      }
6  }
7
8  class Cat extends Animal {
9
10     @Override
11     void sound() {
12         System.out.println(x: "Cat meows");
13     }
14 }
15
16 public class PolymorphismDemo {
17
18     Run | Debug
19     public static void main(String[] args) {
20
21         Animal a = new Cat();
22
23         a.sound();
24     }
25 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER 2
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
C:\Users\hp\OneDrive\Desktop\Alfido_Task1> c: && cd c:\Users\hp\OneDrive\Desktop\Alfido_Task1 && cmd /C ""C:\Program Files\Java\jdk-11.0.10\bin\java.exe -Xlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:61883 --enable-preview -XX:+ShowCodeDetailsInExceptionMessages\Code\User\workspaceStorage\319dfe996e15161d2863a80b8e9d5c75\redhat.java\jdt_ws\Alfido_Task1_18e6317c\bin PolymorphismDemo ""
Cat meows
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
```

Explanation:

This program demonstrates polymorphism through method overriding. The same method behaves differently depending on the object that calls it.

Program 7:

EncapsulationDemo.java

Code:

```
J EncapsulationDemo.java > EncapsulationDemo > main(String[])
1  class StudentInfo {
2
3      private String name;
4
5      public void setName(String name) {
6          this.name = name;
7      }
8
9      public String getName() {
10         return name;
11     }
12 }
13
14 public class EncapsulationDemo {
15
16     Run | Debug
17     public static void main(String[] args) {
18
19         StudentInfo s = new StudentInfo();
20
21         s.setName(name: "Ramsha");
22
23         System.out.println(s.getName());
24     }
25 }
```

Output:

```
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
C:\Users\hp\OneDrive\Desktop\Alfido_Task1> c: && cd c:\Users\hp\OneDrive\Desktop\Alfido_Task1 && cmd /C ""C:\Program Files\Java
lib:jdpw=transport=dt_socket,server=n,suspend=y,address=localhost:60796 --enable-preview -XX:+ShowCodeDetailsInExceptionMessage
g\Code\User\workspaceStorage\319dfe996e15161d2863a80b8e9d5c75\redhat.java\jdt_ws\Alfido_Task1_18e6317c\bin EncapsulationDemo "
Ramsha
C:\Users\hp\OneDrive\Desktop\Alfido_Task1>
```

Explanation:

This program demonstrates encapsulation. The variable is declared private and accessed through public getter and setter methods, ensuring controlled access to data.

OOP Concept Used

1. Class

A class is a blueprint or template used to create objects. It defines the attributes (data) and behaviors (methods) that an object will have. Classes help in organizing code and make programs easier to manage and maintain.

Example: Student, Animal, Dog

2. Object

An object is an instance of a class. It represents a real-world entity and can access the variables and methods defined in the class. Multiple objects can be created from a single class, each having its own set of values.

Example: A student named Alquama can be represented as an object of the Student class.

3. Inheritance

Inheritance is a feature of OOP that allows one class to inherit properties and methods from another class. The class that inherits is called the child class, and the class being inherited from is called the parent class.

Inheritance promotes code reusability and reduces duplication of code.

Example: The Dog class inherits from the Animal class.

4. Polymorphism

Polymorphism means "many forms." It allows the same method to perform different actions depending on the object that invokes it. This makes programs more flexible and easier to extend.

In this task, polymorphism was demonstrated using method overriding, where a child class provides its own implementation of a method already defined in the parent class.

5. Encapsulation

Encapsulation is the process of binding data and methods together into a single unit and restricting direct access to data. It helps protect the internal state of an object from unintended modifications.

In Java, encapsulation is achieved by declaring variables as private and accessing them through public getter and setter methods.

Benefits of Encapsulation:

- Improves data security
- Prevents unauthorized access
- Makes code easier to maintain
- Enhances data integrity

Advantages of OOP

Object-Oriented Programming offers several advantages:

- Encourages code reusability through inheritance.
- Improves code organization and readability.
- Makes applications easier to maintain and update.
- Enhances data security through encapsulation.
- Supports real-world modeling using classes and objects.
- Reduces code duplication and increases efficiency.

Learning Outcomes

- Learned the basics of Java programming.
- Understood the concepts of Object-Oriented Programming.
- Implemented inheritance, polymorphism, and encapsulation in Java.
- Improved logical thinking and problem-solving skills.
- Gained practical experience in writing and executing Java programs.

Conclusion

Through the completion of this task, I gained a strong understanding of Java programming fundamentals and Object-Oriented Programming concepts. I successfully implemented programs based on conditions, loops, arrays, classes, objects, inheritance, polymorphism, and encapsulation.

This task helped me improve my logical thinking and problem-solving abilities while providing hands-on experience in writing and executing Java programs. I learned how OOP concepts help in developing structured, reusable, and maintainable code.

Working on these programs enhanced my confidence in Java programming and gave me practical exposure to important concepts that are widely used in software development. The knowledge gained from this task will serve as a foundation for building more advanced Java applications in the future.

Overall, this task was a valuable learning experience that strengthened my programming skills and deepened my understanding of Object-Oriented Programming principles.

References

- Oracle Java Documentation
- VS Code Documentation
- Java Learning Resources